Emporiqa: AI Chatbot for PrestaShop

Emporiqa for PrestaShop 8.1+ and 9 is AI chat that sells before the purchase and serves after it. Before the purchase, shoppers describe what they need or upload a photo, and it finds matching products from your catalog, compares them and adds them to the cart. After it, it answers questions from your CMS pages and tells customers where their order is. A signed-in customer sees their own prices, as your shop calculates them, and is greeted by name. When a shopper needs a person, your team takes over the chat. It answers in 65+ languages. This module syncs your product catalog and CMS pages to Emporiqa, embeds the chat widget on your storefront, and answers the chat's cart and order status requests.

[image]

- **Integration overview**: emporiqa.com/integrations/prestashop/
- **Full documentation**: emporiqa.com/docs/prestashop/ (webhook format reference, hook examples, troubleshooting)
- **Features**: emporiqa.com/features/ · **FAQ**: emporiqa.com/faq/ · **Pricing**: emporiqa.com/pricing/
- **Live demo**: demo.emporiqa.com and a 30-second video. That demo sells electronics, and the behavior is the same on any catalog.

Requirements

- PrestaShop 8.1+ or 9.x (tested on 8.1 to 9.2)
- PHP 8.0+ on PrestaShop 8.1, PHP 8.1+ on PrestaShop 9. PrestaShop 8.1 also runs on PHP 7.x; this module does not
- An Emporiqa account. Sign up with no card; $25 of signup credit

(~100 free conversations) auto-applied

Installation

1. Get the module from the PrestaShop Addons Marketplace (paid) or from GitHub (free).
2. In your PrestaShop back office, go to **Modules > Module Manager > Upload a Module** and upload `emporiqa.zip`.
3. Click **Configure** on the Emporiqa module.
4. Click **Connect to Emporiqa**. A new tab opens on emporiqa.com. Create a free account (no card required, $25 of signup credit) or sign in if you already have one, then pick the store you want to connect (or create a new one). The module is connected when you return.
5. On the **Sync** tab, click **Send my catalog** and keep the tab open until the progress bar is full. Products, pages, and combinations flow through; the widget appears on your storefront when the first product arrives.

On HTTP, or prefer to paste credentials yourself? Expand **Edit credentials manually** on the Configure page. Paste a **Store ID** and **Connection Secret** from your Emporiqa dashboard under **Settings → Integration**. Both flows reach the same place.

Order status in the chat. Once connected, the Configure page shows a **Ready-made rules** section. Click **Open in Emporiqa** next to **Order status**, then in Emporiqa click **Try it** to test the rule and **Go live** to switch it on. You do not need to copy anything: Emporiqa fills in your shop's address by itself when you connect in one click. If it ever asks for the address, the **Order status address** field in the same section shows the one to use, with a **Copy** button. The rule answers "Where is my order?" from your PrestaShop orders: a shopper signed in to your shop only gives the order reference, and a guest also gives the order's email. Once the order is proven, the chat can tell the shopper everything their order page shows: status, tracking, the items with their prices, the totals, payment, carrier and delivery time, and the delivery and invoice addresses. When the shopper is signed in, Emporiqa can also ask your shop who they are: the name and email on their account and their 10

newest orders (number, date, status, total) in the shops you sync, so the chat can answer "Where is my order?" with their latest order without asking for a number, and fill in their email in your rules. Only the signed-in shopper's own account and orders are sent, never another customer's, a guest order or an address. The other ready-made rules in Emporiqa (**Settings > Rules**) pass returns, cancellations, order changes, quotes and invoice requests to your team by email. If you connected with manual credentials and the section does not show yet, click **Test Connection** on the Sync tab. Shops that already set up the older order tracking address keep it working as before; once Order status is on, the note under **Advanced** tells you how to switch the old one off.

Configuration

All settings are managed from the module configuration page (**Modules > Emporiqa > Configure**):

Connection Settings

The recommended path is **Connect to Emporiqa** (one-click handshake, no credentials to paste). For HTTP sites or manual setup, expand **Edit credentials manually**:

Setting	Description	Default
Store ID	Your Emporiqa store identifier (filled automatically by one-click connect)	(none)
Connection Secret	HMAC-SHA256 signing secret (filled automatically by one-click connect)	(none)

Shops and languages

Setting	Description	Default

Shops (multistore only)	Shops whose catalog is synced and where the chat is shown. All shops share one Emporiqa store; each shop is a channel	All active shops
Languages	Languages included in sync payloads; their pages show the chat. Pages in an unticked language show no chat	All active shop languages

Ready-made rules

Setting	Description	Default
Order status address	The address Emporiqa calls for the Order status rule. Emporiqa fills it in by itself; copy it only if Emporiqa asks for it	auto-generated
Order tracking	The older way to answer "Where is my order?", replaced by the Order status rule. It keeps working for shops that already use it, and sits under **Advanced** as *Old order tracking* once ready-made rules show	On

Advanced

Setting	Description	Default
Auto-sync products	Send each product change as it is saved	On
Auto-sync pages	Send each CMS page change as it is saved	On
Webhook URL	Emporiqa webhook endpoint	`https://emporiqa.com/webhooks/sync/`
Batch Size	Products/pages per webhook request during bulk sync	25

In-chat cart operations are always enabled. No configuration needed.

AI disclosure

The chat's default greeting tells the shopper it is the store's AI assistant, in every language the chat speaks. A custom greeting must keep that disclosure; one that drops it is refused when you save it. Section 8.6 of the Emporiqa Terms treats removing the disclosure, including through custom CSS or custom code, as a breach.

Keeping your catalog in sync

The module pushes product, page, and order changes to Emporiqa automatically as they happen via PrestaShop hooks. Per-product changes such as promos and other specific prices (SpecificPrice), catalog price rule edits, image edits, and combination edits re-emit the affected product on their own. A promo with start or end dates is re-sent when it starts and when it ends: storefront visits check for that at most every 15 minutes, up to 100 products per visit, oldest change first, the next visit continuing with the rest. A catalog price rule covering more than 100 products logs a request for a manual sync instead; pure stock/out-of-stock changes emit a compact availability-only update instead of rebuilding the whole product.

Some changes affect the whole catalog (category or brand renames, currency rate refreshes, tax-rate or tax-rules-group edits, cart-rule changes, new languages enabled). Running a synchronous per-product re-sync from those hooks would block the admin request, so the module logs an actionable warning in **Advanced Parameters → Logs** instead and leaves the catalog refresh to a manual run.

Re-run a full sync from the **Sync** tab when:

- You see one of the "catalog-wide change" warnings in the PrestaShop log
- You add a new shop in multi-shop mode (existing products won't

carry the new shop's data until something else touches them)
- You import products in bulk from a CSV file (PrestaShop sometimes bypasses standard save hooks during bulk imports)
- A custom script, migration, or another module writes catalog data directly to the database
- Emporiqa was unreachable for an extended period (network outage, planned maintenance, expired credentials)

As a safety net, run a full sync once a week to catch any drift that may have built up from background failures.

Product payload fields

Beyond the fields shown in the webhook payload reference, the full product and combination payload carries these PrestaShop-native merchandising and pricing fields:

- `condition`: string or null; PrestaShop's product `condition` (`"new"`, `"used"`, or `"refurbished"`).
- `is_virtual`: boolean; true for digital products with no shipping.
- `available_for_order`: boolean; false for display-only / catalog-mode products. The assistant still describes these but won't add them to the cart.
- `max_order_quantities`: per-channel dict (`{channel: int | null}`) of the maximum allowed per-order quantity. PrestaShop has no native per-order maximum, so this currently always ships `null` (no limit). The field is included for cross-platform contract parity, so a future custom source can populate it.
- `tier_prices`: per-currency list of quantity-based volume discounts (`[{min_quantity, price}]`), present on a price entry only when the product or combination has PrestaShop quantity discounts configured. Each tier reflects the public (guest) shopper's unit price at that break, with the precision the cart multiplies by (6 decimals, or the cent when the shop rounds each item), so the price for a quantity matches the cart total. Group-, customer-, or country-restricted (B2B) tiers are intentionally excluded.

These flags are part of the full product and combination payload, not the lightweight `product.availability` event, which carries only the identification number, SKU, per-channel availability statuses, and stock quantities.

A combination carries no `descriptions`, `categories`, `brands`, `variation_attributes` or `is_parent`: Emporiqa takes the first three from its product and ignores them on a combination.

A product is sent for the shops where it is active and not hidden with **Visibility: Nowhere**; setting a product to Nowhere removes it from Emporiqa. On a multistore install, a combination is sent only for the shops that sell it, and the product's stock in a shop counts only those combinations. The `links` are the product's own URL in each shop and language, built as your storefront builds it (with the category, when your product URL format has one).

Module structure

```
emporiqa/
├── emporiqa.php                    # Main module class (hoo
├── config.xml                      # Module metadata
├── logo.png                        # Module icon
├── classes/
│   ├── EmporiqaActionEndpoint.php     # Ready-made rules
│   ├── EmporiqaCartApiEndpoint.php    # Cart API body (PO
│   ├── EmporiqaCartHandler.php        # In-chat cart oper
│   ├── EmporiqaChannelResolver.php    # Multi-shop → chan
│   ├── EmporiqaConnectHandshake.php   # One-click connect
│   ├── EmporiqaConnectNonce.php       # One-click connect
│   ├── EmporiqaCustomerPrices.php     # What a signed-in
│   ├── EmporiqaCustomerInfo.php       # Who a signed-in c
│   ├── EmporiqaJsonResponse.php       # The one JSON resp
│   ├── EmporiqaLanguageHelper.php     # Language mapping
│   ├── EmporiqaOrderFormatter.php     # Order payload for
│   ├── EmporiqaOrderStatus.php        # Order status look
│   ├── EmporiqaOrderTrackingEndpoint.php # Order trackin
```

```
│   ├── EmporiqaPageFormatter.php     # CMS page payload
│   ├── EmporiqaProductFormatter.php  # Product/combinati
│   ├── EmporiqaSchema.php            # Module tables, cr
│   ├── EmporiqaSignatureHelper.php   # HMAC-SHA256 signi
│   ├── EmporiqaSyncService.php       # Bulk sync orchest
│   ├── EmporiqaTokenEndpoint.php     # Signed customer t
│   └── EmporiqaWebhookClient.php     # HTTP client for w
├── controllers/
│   ├── admin/
│   │   ├── AdminEmporiqaController.php        # Admin me
│   │   └── AdminEmporiqaConnectController.php # One-clic
│   └── front/
│       ├── action.php                # Ready-made rules
│       ├── cartapi.php               # Cart API endpoint
│       ├── ordertracking.php         # Order tracking en
│       └── token.php                 # Customer token en
├── views/
│   ├── css/admin.css                 # Admin configurati
│   ├── img/                          # Module images (re
│   ├── js/
│   │   ├── admin-sync.js            # Bulk sync UI with
│   │   ├── front-cart-handler.js    # Chat widget cart i
│   │   └── front-customer-token.js  # Hands the customer
│   └── templates/
│       ├── admin/configure.tpl       # Configuration pag
│       ├── admin/sync_tab.tpl        # Sync tab template
│       ├── admin/sync_health_line.tpl # Last full sync l
│       └── hook/header.tpl           # Widget embed (dis
├── translations/fr.php               # French back-offic
└── upgrade/                          # Version upgrade s
```

Registered PrestaShop hooks

Hook

displayHeader

actionProductSave
actionProductDelete
actionObjectCombination{Add,Update,Delete}After
actionObjectCms{Add,Update,Delete}After
actionValidateOrder
actionOrderStatusPostUpdate
actionUpdateQuantity
actionProductOutOfStock
actionObjectSpecificPrice{Add,Update,Delete}After

`actionObjectSpecificPriceRule{Update,Delete}Before`

`actionAdminSpecificPriceRuleController{Delete,Bulkdelete}Before`
`actionObjectImage{Add,Update,Delete}After`
`actionObjectCategory{Update,Delete}After`
`actionObjectManufacturer{Update,Delete}After`
`actionObjectCartRule{Add,Update,Delete}After`
`actionObjectCurrencyUpdateAfter`
`actionObjectTaxUpdateAfter` / `actionObjectTaxRulesGroupUpdateAfter`

`actionObjectLanguageAddAfter`

Extensibility hooks

Developers can hook into the sync pipeline to customize payloads or cancel syncs:

Hook	Purpose	Key Param
`actionEmporiqaFormatProduct`	Modify product/ variation payload before sending	`&$data`, `$product` `$event_t`
`actionEmporiqaFormatPage`	Modify page payload before sending	`&$data`, `$` `$event_t`
`actionEmporiqaFormatOrder`	Modify the `order.completed` event payload	`&$data`, `$`
`actionEmporiqaShouldSyncProduct`	Conditionally cancel a product sync	`$product` `$event_t` `&$should`
`actionEmporiqaShouldSyncPage`	Conditionally cancel a page sync	`$page`, `$event_t` `&$should`
`actionEmporiqaWidgetParams`	Modify chat widget embed parameters	`&$params`
`actionEmporiqaOrderStatus`	Modify the Order status ready-made rule's answer, or add your own fields under `extra`	`&$data`, `$`

`actionEmporiqaOrderTracking`	Modify the order tracking response	`&$data, $`
`actionEmporiqaCustomerInfo`	Modify what Emporiqa learns about a signed-in customer (account name and email, newest orders), or add your own fields under `extra`	`&$data, $custome`

`actionEmporiqaOrderStatus` runs after the module has filled the answer (status, tracking, items, totals, payment, carrier, addresses), so you can change any of it. Put fields of your own under `extra`, which the chat reads when the shopper asks for them: string keys, values that are strings, numbers, booleans or nested lists and objects, at most 30 keys, 3 levels deep and 500 characters a string. Other keys you add are ignored.

```
public function hookActionEmporiqaOrderStatus(array $para
{
    $order = $params['order'];
    $params['data']['extra']['gift_message'] = 'Happy bir
    $params['data']['extra']['warehouse'] = 'Lyon';
}
```

`actionEmporiqaCustomerInfo` runs after the module has filled `$params['data']`: `customer` (`name`, `first_name`, `last_name`, `email` from the account) and `orders` (up to 10, newest first: `order_number`, `placed_at`, `status_code`, `status_label`, `total`, `currency`). Remove what you do not want Emporiqa to know, or add fields under `extra` with the same limits as above.

```
public function hookActionEmporiqaCustomerInfo(array $par
{
    $customer = $params['customer'];
    unset($params['data']['customer']['email']);
    $params['data']['extra']['loyalty_points'] = 120;
```

```
}
```

Pricing

The module is paid on PrestaShop Addons and free on GitHub. The Emporiqa service itself is pay-as-you-go: $0/month base + $0.25/conversation, with $25 of signup credit (about 100 conversations) and no card required at signup. You set a monthly spending limit on the billing page and can raise, lower or remove it. Voice mode (the shopper speaks and hears the answer read aloud) is optional and off by default: a conversation where the shopper speaks costs $0.15 more, charged once, and counts toward that limit. Prices exclude VAT. Enterprise option for catalogs over 100,000 products. Full pricing at emporiqa.com/pricing/.

Known issues

- The messages written by the **Sync** and **Test Connection** buttons, and the one-click connect error messages, are in English only. The rest of the configuration page is available in English and French.
- The controllers under `controllers/` and `emporiqa.php` keep PHP 7 syntax (no trailing commas in calls or parameters), so a shop left on PHP 7 gets a clean error instead of a fatal one; the PHP 8 code lives in `classes/`.
- The full list, with technical detail, is in CHANGELOG.md.

Support

Email support@emporiqa.com.

License

Academic Free License 3.0 (AFL-3.0)